

Licence 2^{ème} Année

Sciences pour l'ingénieur (SPI)

UE 3 – Informatique

Programmation

Chapitre 1 :

Premiers pas en

Programmation

Enseignant : Yves André CHAPUIS

1. Les premières notions

1.1. Les cinq types de langage de programmation

Les langages de programmation peuvent être classés selon plusieurs critères, mais généralement, on distingue **cinq grands types** de langages en fonction de leur paradigme (c'est-à-dire leur manière de concevoir et d'organiser le code).



Voici la description de ces cinq types de langage de programmation :

1. Langages impératifs (procedural programming language)

- Le programme décrit comment faire les choses, étape par étape (séquences d'instructions).
- C#, Java, Pascal, Fortran, ...
- Utilisation de variables, boucles, conditions ; proche du fonctionnement de la machine.

2. Langages orientés objet (object-oriented programming language)

- Le programme est structuré autour d'**objets** (entités qui combinent données et comportements).
- Python (multi-paradigme), C++, Java, ...
- Notions de classe, héritage, encapsulation, polymorphisme.

3. Langages fonctionnels (functional programming language)

- Le programme est basé sur l'**évaluation de fonctions mathématiques** et évite les états mutables et effets de bord.
- Haskell, SML, Scala (multi-paradigme), ...
- Programmation déclarative, fonctions comme objets de première classe, récursivité.

4. Langages logiques (logic programming language)

- Le programme spécifie **ce qui doit être vrai**, et le moteur d'exécution déduit la réponse.
- Prolog, Datalog, ...
- Basés sur la logique formelle (prédicats, règles), utilisés en intelligence artificielle et systèmes experts.

5. Langages de script (scripting programming language)

- Souvent utilisés pour **automatiser des tâches** simples ou manipuler des logiciels existants.
- JavaScript, PHP, Perl, Ruby, ...
- Interprétés, syntaxe souple, utiles pour le prototypage rapide,

Remarque : Certains langages comme **Python** ou **Scala** sont **multi-paradigmes** : ils supportent plusieurs styles de programmation (impératif, objet, fonctionnel, etc.).

1.2. Pourquoi choisir le langage de programmation Python ?

Le langage de programmation Python a été créé en **1989** par *Guido van Rossum* (voir figure ci-dessous), aux Pays-Bas. Le nom Python vient d'un hommage à la série télévisée *Monty Python's Flying Circus* dont *G. van Rossum* est fan. La première version publique de ce langage a été publiée en **1991**.



Guido van Rossum (Pays-Bas) créateur de Python en 1989 et accessible au public dès 1991

La dernière version de Python est la **version 3**. Plus précisément, la version 3.7 a été publiée en juin 2018. La version 2 de Python est désormais obsolète et cessera d'être maintenue après le 1er janvier 2020. Dans la mesure du possible évitez de l'utiliser.

La Python Software Foundation est l'association qui organise le développement de Python et anime la communauté de développeurs et d'utilisateurs.

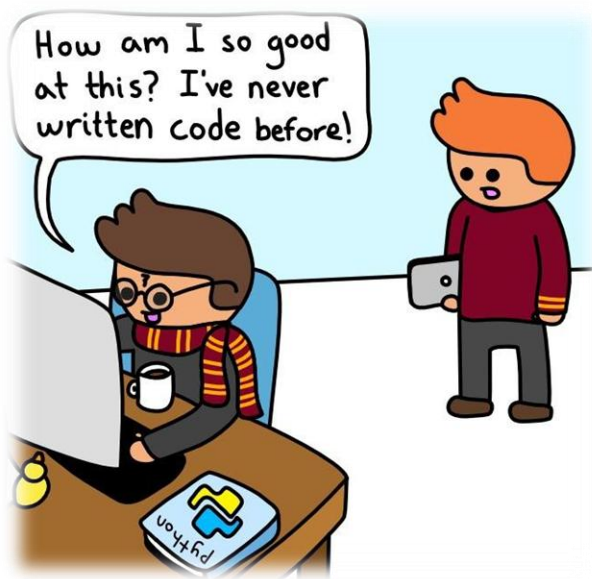
Ce langage de programmation présente de nombreuses caractéristiques intéressantes :

- Il est **multiplateforme**. C'est-à-dire qu'il fonctionne sur de nombreux systèmes d'exploitation : Windows, Mac OS X, Linux, Android, iOS, depuis les mini-ordinateurs Raspberry Pi jusqu'aux supercalculateurs.

- Il est **gratuit**. Vous pouvez l'installer sur autant d'ordinateurs que vous voulez (même sur votre téléphone !).
- C'est un **langage de haut niveau**. Il demande relativement peu de connaissance sur le fonctionnement d'un ordinateur pour être utilisé.
- C'est un **langage interprété**. Un script Python n'a pas besoin d'être compilé pour être exécuté, contrairement à des langages comme le C ou le C++.
- Il est **orienté objet**. C'est-à-dire qu'il est possible de concevoir en Python des entités qui miment celles du monde réel (une cellule, une protéine, un atome, etc.) avec un certain nombre de règles de fonctionnement et d'interactions.
- Il est relativement **simple** à prendre en main.
- Enfin, il est très utilisé dans de nombreux domaines des **sciences** et de l'**ingénierie**, et récemment en **analyse de données** et **intelligence artificiel**. Toutes ces caractéristiques font que Python est désormais enseigné dans de nombreuses formations, depuis l'enseignement secondaire jusqu'à l'enseignement supérieur.

1.3. Comment programmer en Python ?

Pour apprendre la programmation Python, il va falloir que vous pratiquiez et pour cela il est préférable que Python soit installé sur votre ordinateur. La bonne nouvelle est que vous pouvez installer gratuitement Python sur votre machine, que ce soit sous Windows, Mac OS X ou Linux. Nous donnons dans cette rubrique un résumé des points importants concernant cette installation.



1.3.1. Python 2 ou Python 3 ?

Ce cours est basé sur la version 3 de Python, qui est désormais le standard. Si, néanmoins, vous deviez un jour travailler sur un ancien programme écrit en Python 2, sachez qu'il existe quelques différences importantes entre Python 2 et Python 3.

1.3.2. Editer de texte

L'apprentissage d'un langage informatique comme Python va nécessiter d'écrire des lignes de codes à l'aide d'un éditeur de texte.

Si vous êtes débutants, on vous conseille d'utiliser :

- notepad++ sous Windows ;
- BBEdit ou CotEditor sous Mac OS X ;
- gedit sous Linux.

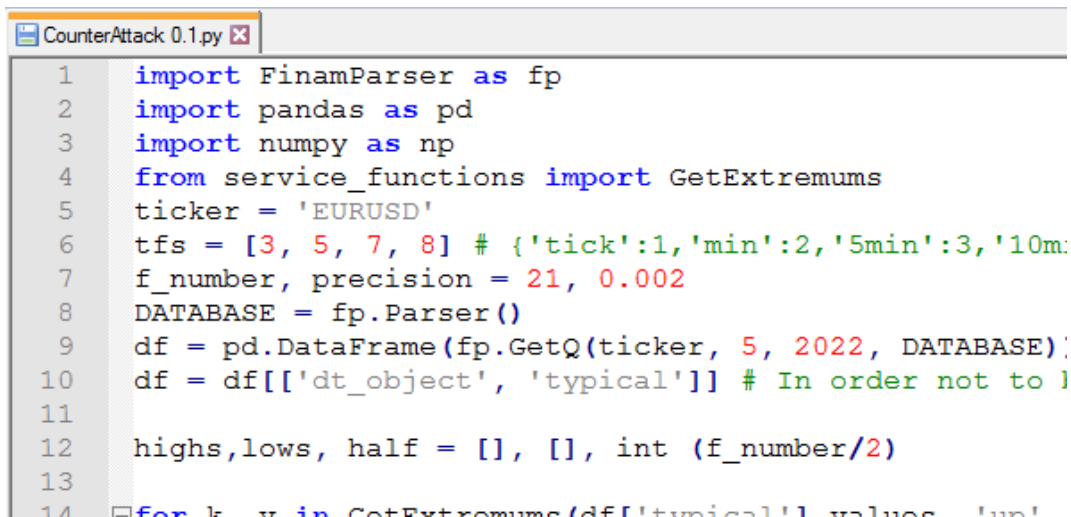
La configuration de ces éditeurs de texte est détaillée dans la rubrique Installation de Python disponible en ligne.

Pour les étudiants déjà initiés à Python, vous préférez sûrement d'autres éditeurs comme :

- Atom ;
- Visual Studio Code ;
- Sublime Text ;
- Emacs ;
- Vim ;
- Geany ;
- ...

Remarque

L'extension de fichier standard des scripts Python est .py



```

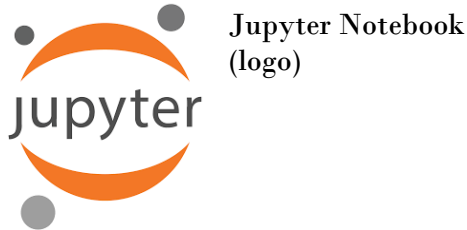
1  import FinamParser as fp
2  import pandas as pd
3  import numpy as np
4  from service_functions import GetExtremums
5  ticker = 'EURUSD'
6  tfs = [3, 5, 7, 8] # {'tick':1,'min':2,'5min':3,'10m':4}
7  f_number, precision = 21, 0.002
8  DATABASE = fp.Parser()
9  df = pd.DataFrame(fp.GetQ(ticker, 5, 2022, DATABASE))
10 df = df[['dt_object', 'typical']] # In order not to lose
11
12 highs, lows, half = [], [], int (f_number/2)
13
14 for k, v in GetExtremums(df[['typical']].values.tolist()):

```

À toute fin utile, on rappelle que les logiciels Microsoft Word, WordPad et LibreOffice Writer ne sont pas des éditeurs de texte, ce sont des traitements de texte qui ne peuvent pas et ne doivent pas être utilisés pour écrire du code informatique.

1.4. Environnement « Jupyter Notebook » de programmation

Pour l'ensemble des enseignements de la matière, les programmes seront développés sous dans l'environnement **Jupyter Notebook**. On parlera de « Notebook Jupyter » lorsque l'on développera un fichier de programmation Python.



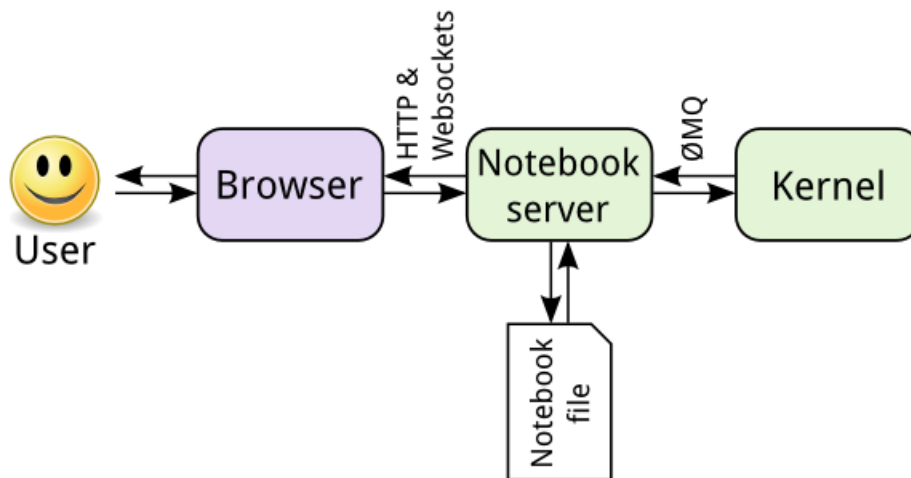
1.4.1. C'est quoi Jupyter Notebook ?

Quiconque travaille ou a des collègues dans le domaine de la **science des données** doit avoir entendu parler de **Jupyter Notebook**, une application Web interactive qui permet la création et le partage de documents avec du code dynamique.

Le produit est largement utilisé dans les domaines de l'exploration de données. Il facilite les activités de visualisation, de nettoyage et d'exploration de données, en plus de permettre le mélange d'extraits de code et de texte, optimisant la création de présentations et de rapports.

Finalement, **Jupyter Notebook** permet la construction de tout en un seul endroit, comme s'il s'agissait d'un **IDE (environnement de développement intégré)** pour le **data scientist**.

L'image suivante montre une architecture de base de l'outil.



Architecture de base de l'application Web interactive **Jupyter Notebook**

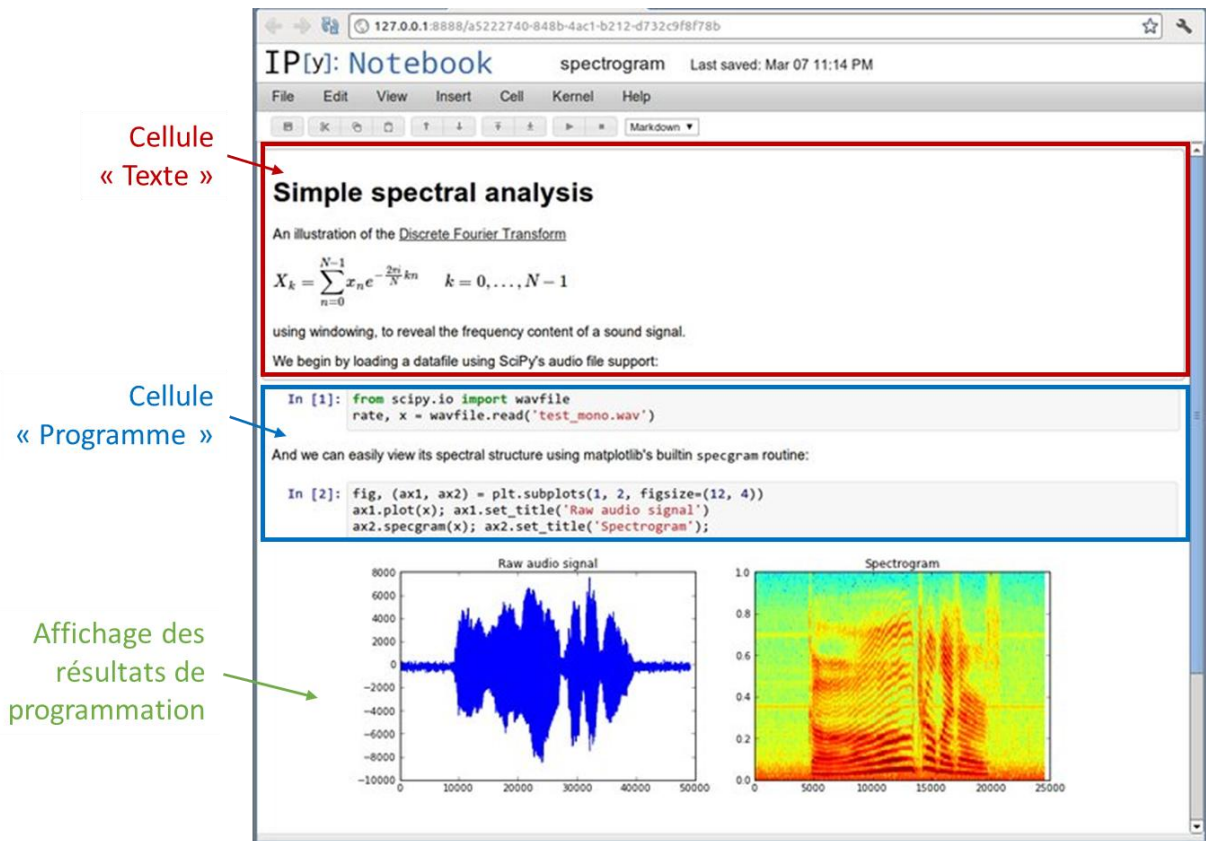
1.4.2. Quels avantages y a-t-il à utiliser un Notebook Jupyter ?

Les Notebooks Jupyter sont particulièrement utiles pour présenter le travail effectué par votre équipe data, notamment grâce à une combinaison de code, de Markdown (éditeur de texte), de liens et d'images.

Voici ci-dessous un exemple de **Notebook Jupyter** dans le cas de la programmation de calculs et de visualisation d'équations différentielles de Lorentz.

On notera que le Notebook Jupyter est représenté par des cellules où l'on peut travailler indépendamment, comme le montre la figure ci-dessous.

- Texte (sections, sous-section, explication, commentaires, etc.)
- Programme (code Python)
- Résultats (après exécution - graphe, calcul, etc.)



Représentation du Notebook Jupyter structuré en cellules, permettant un travail indépendant dans chacune d'elles : Texte, programme et résultats.

1.5. Comment accéder à « Jupyter Notebook » ?

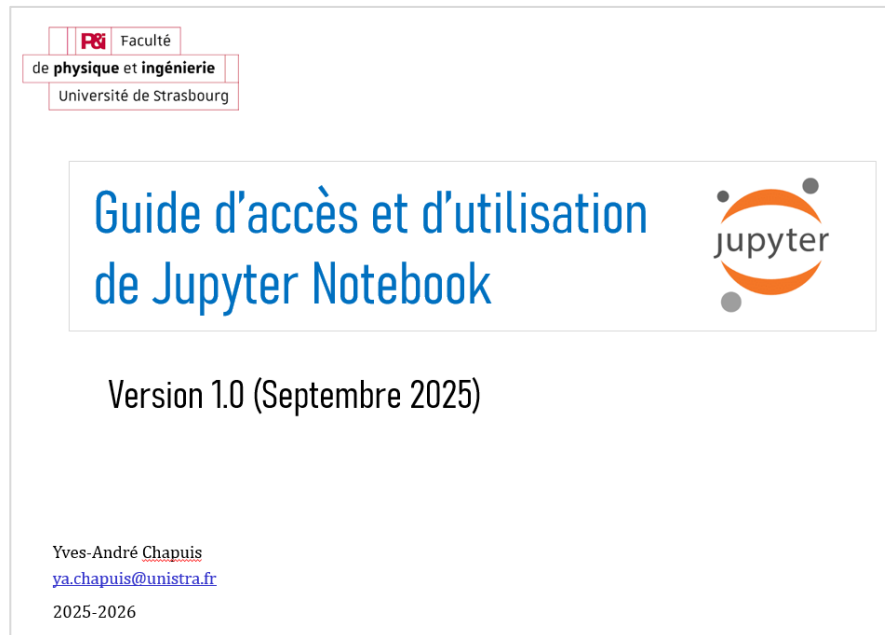
Un guide d'accès a été créé pour vous permettre d'utiliser Jupyter Notebook. Il sera distribué en version papier en cours et déposé sur Moodle.

Dans ce guide vous trouverez les informations suivantes :

1. Solutions d'utilisation de Jupyter Notebook
2. Jupyter Notebook sur les postes informatiques de la Faculté de Physique et Ingénierie
3. Utilisation à distance de Jupyter Notebook (HORS campus Unistra)

Egalement, le guide vous informe comment installer Jupyter Notebook sur votre ordinateur PC.

Couverture « Guide d'accès et d'utilisation de Jupyter Notebook » :



1.6. Premier programme Python

1.6.1. Interpréteur Python

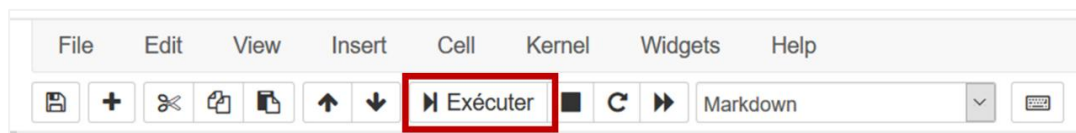
Python est un **langage interprété**, c'est-à-dire que chaque ligne de **programme** ou **script** en Python est lue puis interprétée afin d'être exécutée par l'ordinateur.

Pour nous rendre compte de l'action de l'interpréteur, nous allons exécuter le script Python de la section précédent. L'exécution va lancer l'interpréteur Python. Nous devrions obtenir quelque chose de ce style :

1. Ecrire dans une cellule d'un Notebook Jupyter, le script suivant :

```
1 sous cette forme,  
2 sur plusieurs lignes,  
3 pour les éléments les plus longs.
```

2. Cliquer sur « Exécuter » dans le menu Jupyter



Cela va lancer l'interpréteur Python

3. Résultat d'exécution affiché

```
File "<ipython-input-1-d911cf5dfabd>", line 1  
    sous cette forme,  
    ^  
SyntaxError: invalid syntax
```


On note que l'interpréteur Python a détecté une erreur de syntaxe : `SyntaxError : invalid syntax` dès la première ligne de code.

C'est normal ! La ligne 1 n'est pas une syntaxe correcte de Python (les lignes suivantes non plus). Nous verrons pourquoi plus tard dans le cours.

L'**interpréteur Python** est donc un système interactif dans lequel vous pouvez entrer un script, que Python exécutera sous vos yeux.

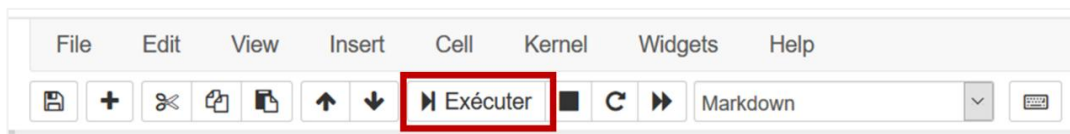
Il existe de nombreux autres langages interprétés comme Perl ou R. Le gros avantage de ce type de langage est qu'on peut immédiatement tester une commande à l'aide de l'**interpréteur**, ce qui est très utile pour **débugger** (c'est-à-dire trouver et corriger les éventuelles erreurs d'un programme). Gardez bien en mémoire cette propriété de Python qui pourra parfois vous faire gagner un temps précieux !

1.6.2. Programmation d'un affichage en Python !

Dans une cellule du Notebook Jupyter, taper l'instruction : `print("Hello world!")`

```
1 print("Hello world!")
```

Puis, sélectionner « **Exécuter** » dans le menu, comme le montre la capture d'écran ci-dessous :



Python va exécuter le code Python directement et affiché le résultat :

```
Hello world!
```

L'interpréteur Python n'a détecté aucune erreur dans l'exécution du script et a pu aller au bout de l'action d'affichage. La syntaxe en Python était donc correcte !

En résumé, voici ce que vous avez exécuté et ce qui a dû apparaître sur votre écran :

```
1 print("Hello world!")
Hello world!
```

Bravo, vous avez codé et exécuté avec succès votre premier programme Python !

1.6.3. Autres exemples de programme Python

Vous pouvez refaire un nouvel essai en vous servant cette fois de l'interpréteur comme d'une **calculatrice** :

```
1 1+1
2
1 6*3
18
```

Python n'a détecté aucune erreur dans l'exécution du script et a pu aller au bout des calculs (le résultat s'affiche par défaut à l'exécution). Comme on le verra plus tard, les opérateurs mathématiques sont également des mêmes opérateurs utilisés en Python.

1.7. Notations élémentaires du script

1.7.1. Numérotation des lignes

Les commandes, les instructions Python, les résultats et les contenus sous Jupyter Notebook sont indiqués avec cette police pour les éléments ponctuels, soit :

```
1 sous cette forme,  
2 sur plusieurs lignes,  
3 pour les éléments les plus longs.
```

Pour ces derniers, le **numéro à gauche** indique le **numéro de la ligne** et sera utilisé pour faire référence à une instruction particulière. Ce numéro n'est bien sûr là qu'à titre indicatif.

Par ailleurs, dans le cas de programmes, de contenus de fichiers ou de résultats trop longs pour être inclus dans leur intégralité, la notation [...] indique une coupure arbitraire de plusieurs caractères ou lignes.

1.7.2. Texte de commentaire

Dans un script, tout ce qui suit le caractère **#** est ignoré par Python jusqu'à la fin de la ligne et est considéré comme un commentaire.

Les commentaires doivent expliquer votre code dans un langage humain. L'utilisation des commentaires est rediscutée dans les « *Bonnes pratiques en programmation Python* » que nous verrons en travaux pratiques.

Voici un exemple de commentaire :

```
1 # Votre premier commentaire en Python.  
2 # Afficher Hello world!  
3 print("Hello world!")  
4  
5 # D'autres commentaires plus utiles pourraient suivre.
```

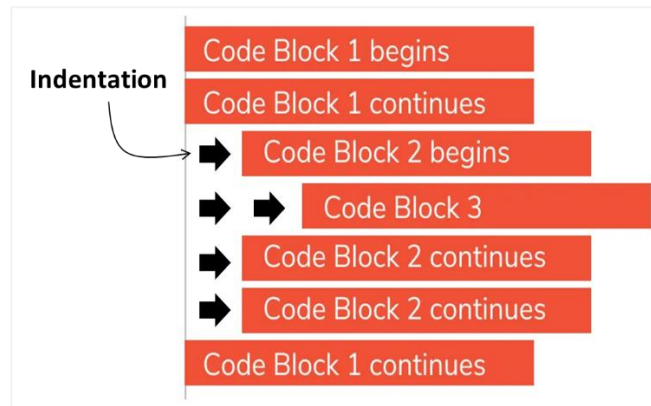
Remarque

On appelle souvent à tort le caractère **#** « dièse ». On devrait plutôt parler de « croisillon ».

1.8. Notion de bloc d'instructions et d'indentation

En programmation, il est courant de répéter un certain nombre de choses (avec le chapitre sur les **boucles**) ou d'exécuter plusieurs instructions si une condition est vraie (avec le chapitre sur les **conditionnels** ou **tests**).

Par exemple, imaginons que nous souhaitons afficher chacune des bases d'une séquence d'ADN, les compter puis afficher le nombre total de bases à la fin. Nous pourrions utiliser l'algorithme présenté en pseudo-code dans la figure ci-dessous.



Notion d'indentation et de bloc d'instructions.

Pour chaque base de la séquence ATCCGACTG, nous souhaitons effectuer deux actions : d'abord afficher la base puis compter une base de plus. Pour indiquer cela, on décalera vers la droite ces deux instructions par rapport à la ligne précédente (pour chaque base [...]). Ce décalage est appelé indentation, et l'ensemble des lignes indentées constitue un bloc d'instructions.

Une fois qu'on aura réalisé ces deux actions sur chaque base, on pourra passer à la suite, c'est-à-dire afficher la taille de la séquence. Pour bien préciser que cet affichage se fait à la fin, donc une fois l'affichage puis le comptage, de chaque base, terminés, la ligne correspondante n'est pas indentée (c'est-à-dire qu'elle n'est pas décalée vers la droite).

Pratiquement, l'indentation en Python doit être homogène (soit des espaces, soit des tabulations, mais pas un mélange des deux). Une indentation avec 4 espaces est le style d'indentation recommandé.

Si tout cela semble un peu complexe, ne vous inquiétez pas ! Vous allez comprendre tous ces détails chapitre après chapitre.

1.9. Autres ressources

Pour compléter votre apprentissage de Python, n'hésitez pas à consulter d'autres ressources complémentaires à cet ouvrage.

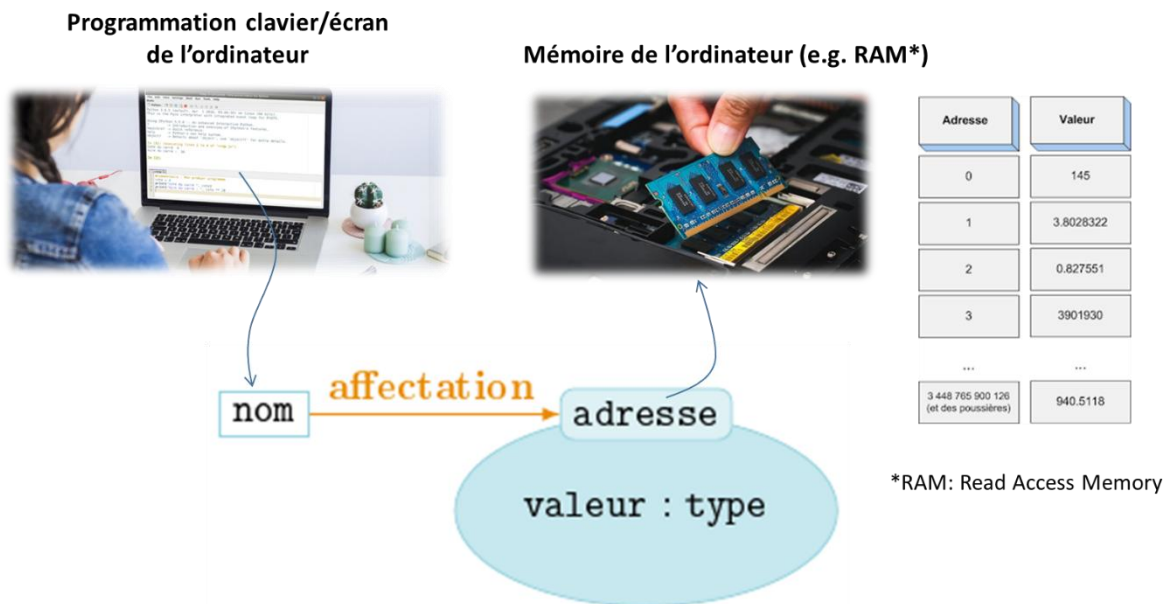
D'autres auteurs abordent l'apprentissage de Python d'une autre manière. Nous vous conseillons les ressources suivantes en langue française :

- Le livre *Apprendre à programmer avec Python 3* de *Gérard Swinnen*. Cet ouvrage est téléchargeable gratuitement sur
- le site de *Gérard Swinnen*. Les éditions *Eyrolles* proposent également la version papier de cet ouvrage.
- Le livre *Apprendre à programmer en Python avec PyZo et Jupyter Notebook* de *Bob Cordeau* et *Laurent Pointal*, publié
- aux éditions *Dunod*. Une partie de cet ouvrage est téléchargeable gratuitement sur le site de *Laurent Pointal*.
- Le livre *Apprenez à programmer en Python* de *Vincent Legoff* que vous trouverez sur le site *Openclassrooms*.
- Et pour terminer, une ressource incontournable en langue anglaise :
- Le site *www.python.org*. Il contient énormément d'informations et de liens sur Python. La page d'index des modules est particulièrement utile (et traduite en français).

2. Variables

2.1. Définition

Une **variable** est une zone de la **mémoire** de l'ordinateur dans laquelle une **valeur** est stockée. Aux yeux du programmeur, cette variable est définie par un **nom**, alors que pour l'ordinateur, il s'agit en fait d'une **adresse**, c'est-à-dire d'une zone particulière de la mémoire, comme le montre la figure ci-dessous.



Affectation d'une variable dans un ordinateur

En Python, la **déclaration** d'une variable et son **initialisation** (c'est-à-dire la première valeur que l'on va stocker dedans) se font en même temps. Pour vous en convaincre, testez les instructions suivantes après avoir lancé l'interpréteur :

```

1 x = 2
2 x
2

```

- **Ligne 1 :**

Dans cet exemple, nous avons déclaré, puis initialisé la **variable x** avec la **valeur 2**. Notez bien qu'en réalité, il s'est passé plusieurs choses :

- Python a « deviné » que la variable était un *entier*. On dit que Python est un langage au typage dynamique.
- Python a alloué (réservé) l'espace en **mémoire** pour y accueillir un *entier*. Chaque type de variable prend plus ou moins d'espace en mémoire. Python a aussi fait en sorte qu'on puisse retrouver la variable sous le **nom** x.
- Enfin, Python a assigné la valeur 2 à la variable x.

Dans d'autres langages (en C par exemple), il faut coder ces différentes étapes une par une. Python étant un langage dit de haut niveau, la simple instruction `x = 2` a suffi à réaliser les 3 étapes en une fois !

• Ligne 2 :

L'interpréteur nous a permis de connaître le contenu de la variable juste en tapant son nom `x`.

Retenez ceci car c'est une spécificité de l'interpréteur Python, très pratique pour chasser (debugger) les erreurs dans un programme. Par contre, la ligne d'un script Python qui contient seulement le nom d'une variable (sans aucune autre indication) n'affichera pas la valeur de la variable à l'écran lors de l'exécution (pour autant, cette instruction reste valide et ne générera pas d'erreur).

Sachez par ailleurs que l'opérateur d'affectation `=` s'utilise dans un certain sens. Par exemple, l'instruction `x = 2` signifie qu'on attribue la valeur située à droite de l'opérateur `=` (ici, `2`) à la variable située à gauche (ici, `x`).

Enfin, dans l'instruction `x = y - 3`, l'opération `y - 3` est d'abord évaluée et ensuite le résultat de cette opération est affecté à la variable `x`.

2.2. Les types de variables

Le type d'une variable correspond à la nature de celle-ci. Les trois principaux types dont nous aurons besoin dans un premier temps sont des :

- Nombres entiers (integer ou int)
- Nombres décimaux (float)
- Chaînes de caractères (string ou str).
- Nombres booléens (boolean)

Bien sûr, il existe de nombreux autres types (par exemple, les *nombres complexes*, etc.).

Si vous n'êtes pas effrayés, vous pouvez vous en rendre compte ici :

<https://docs.python.org/fr/3.7/library/stdtypes.html>

Dans l'exemple précédent, nous avons stocké un *nombre entier* (integer ou int) dans la **variable** `x`, mais il est tout à fait possible de stocker des nombres décimaux (float), des *chaînes de caractères* (string ou str) ou de nombreux autres types de variable que nous verrons par la suite :

<pre>1 y = 3.14 2 y</pre> 3.14	<pre>1 b = 'salut' 2 b</pre> 'salut'
<pre>1 a = "bonjour" 2 a</pre> 'bonjour'	<pre>1 c = ""girafe"" 2 c</pre> 'girafe'
	<pre>1 d = '''lion''' 2 d</pre> 'lion'

Remarque

Python reconnaît certains **types de variable** automatiquement (*entier, décimaux*). Par contre, pour une *chaîne de caractères*, il faut l'entourer de guillemets (doubles, simples, voire trois guillemets successifs doubles ou simples) afin d'indiquer à Python le début et la fin de la *chaîne de caractères*.

Dans l'interpréteur, l'affichage direct du contenu d'une *chaîne de caractères* se fait avec des guillemets simples, quel que soit le type de guillemets utilisé pour définir la *chaîne de caractères*.

En Python, comme dans la plupart des langages de programmation, c'est le point qui est utilisé comme **séparateur décimal**. Ainsi, 3.14 est un nombre reconnu comme un *décimal* en Python alors que ce n'est pas le cas de 3,14.

2.3. Nommage

Le nom des **variables** en Python peut être constitué de lettres minuscules (a à z), de lettres majuscules (A à Z), de nombres (0 à 9) ou du caractère souligné (_). Vous ne pouvez pas utiliser d'espace dans un nom de variable.

Par ailleurs, **un nom de variable ne doit pas débuter par un chiffre** et il n'est pas recommandé de le faire débuter par le caractère _ (sauf cas très particuliers).

De plus, il faut absolument éviter d'utiliser un mot « réservé » par Python comme nom de **variable** (par exemple : print, range, for, from, etc.).

Enfin, Python est sensible à la casse, ce qui signifie que les variables Test, test ou TEST sont différentes.

2.4. Ecriture scientifique

On peut écrire des nombres très grands ou très petits avec des puissances de 10 en utilisant le symbole « e » :

1	1e6
1000000.0	

1	3.12e-3
0.00312	

On appelle cela **écriture ou notation scientifique**. On pourra noter deux choses importantes :

- 1e6 ou 3.12e-3 n'implique pas l'utilisation du nombre exponentiel e mais signifie 1×10^6 ou 3.12×10^{-3} respectivement ;
- Même si on ne met que des entiers à gauche et à droite du symbole e (comme dans 1e6), Python génère systématiquement un float.

Enfin, vous avez sans doute constaté qu'il est parfois pénible d'écrire des nombres composés de beaucoup de chiffres, par exemple le nombre d'Avogadro $6.02214076 \times 10^{23}$ ou le nombre d'humains sur Terre (au 26 août 2020) 7807568245. Pour s'y retrouver, Python autorise l'utilisation du caractère « souligné » (ou underscore) _ pour séparer des groupes de chiffres.

Par exemple :

```
1 avogadro_number = 6.022_140_76e23
2 print(avogadro_number)

6.02214076e+23

1 humans_on_earth = 7_807_568_245
2 print(humans_on_earth)

7807568245
```

Dans ces exemples, le caractère « _ » est utilisé pour séparer des groupes de 3 chiffres mais on peut faire ce qu'on veut :

```
1 print(7_80_7568_24_5)

7807568245
```

2.5. Opérations

2.5.1. Opérations sur les types numériques

Voici les opérateurs mathématiques de base en Python, utilisés pour effectuer des calculs arithmétiques :

Opérateur	Symbole	Description	Exemple (a = 10 , b = 3)
Addition	+	Addition de deux valeurs	a + b → 13
Soustraction	-	Soustraction	a - b → 7
Multiplication	*	Multiplication	a * b → 30
Division	/	Division (résultat float)	a / b → 3.333...
Division entière	//	Division entière (quotient)	a // b → 3
Modulo	%	Reste de la division entière	a % b → 1
Puissance	**	Exponentiation	a ** b → 1000

Exemple d'utilisation des opérateurs mathématiques de base en Python :

```
a = 10
b = 3

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.333...
print(a // b)   # 3
print(a % b)    # 1
print(a ** b)   # 1000
```

Pour obtenir le **quotient** et le **reste** d'une division entière, on utilise respectivement les symboles `//` et modulo `%`.

Petit rappel sur la division entière :

https://fr.wikipedia.org/wiki/Division_euclidienne

Remarque

Si on mélange les types *entiers* et *décimaux*, le résultat est renvoyé comme un float (car ce type est plus général). Par ailleurs, l'utilisation de parenthèses permet de gérer les priorités.

L'opérateur `/` effectue une division. Contrairement aux opérateurs `+`, `-` et `*`, celui-ci renvoie systématiquement un float :

```
1 3 / 4
0.75
```

```
1 2.5 / 2
1.25
```

L'opérateur puissance utilise les symboles `**` :

```
1 2 ** 3
8
```

```
1 2 ** 4
16
```

2.5.2. Opérateurs combinés

Enfin, il existe des opérateurs « combinés » qui effectue une opération et une affectation en une seule étape :

1	i = 0
2	i = i + 1
3	i
1	
1	i += 1
2	i
2	
1	i += 2
2	i
4	

L'opérateur += effectue une addition puis affecte le résultat à la même variable. Cette opération s'appelle une « **incrément** ».

Les opérateurs -=, *= et /= se comportent de manière similaire pour la soustraction, la multiplication et la division.

2.5.3. Opérations sur les chaînes de caractères

Pour les chaînes de caractères, 2 opérations sont possibles, l'addition et la multiplication :

1	chaine = "Salut"
2	chaine
	'Salut'
1	chaine + "Python"
	'SalutPython'
1	chaine * 3
	'SalutSalutSalut'

L'opérateur d'addition + **concatène** (assemble) deux chaînes de caractères. L'opérateur de multiplication * entre un nombre entier et une chaîne de caractères **duplique** (répète) plusieurs fois une chaîne de caractères.

Attention

Vous observez que les opérateurs + et * se comportent différemment s'il s'agit d'entiers ou de chaînes de caractères : 2 + 2 est une *addition* alors que "2" + "2" est une **concaténation**. On appelle ce comportement redéfinition des opérateurs.

2.5.4. Opérations illicites

Attention à ne pas faire d'opération illicite car vous obtiendriez un message d'erreur :

```
1 "toto" * 1.3

-----

TypeError                                Traceback (most recent call last)
<ipython-input-32-d61e3a6c92fe> in <module>
----> 1 "toto" * 1.3

TypeError: can't multiply sequence by non-int of type 'float'
```

```
1 "toto" + 2

-----

TypeError                                Traceback (most recent call last)
<ipython-input-33-9e99e395f7eb> in <module>
----> 1 "toto" + 2

TypeError: can only concatenate str (not "int") to str
```

Notez que Python vous donne des informations dans son message d'erreur. Dans le second exemple, il indique que vous devez utiliser une variable de type `str` c'est-à-dire une *chaîne de caractères* et pas un `int`, c'est-à-dire un *entier*.

2.6. Conversions de types

En programmation, on est souvent amené à convertir les types, c'est-à-dire passer d'un type numérique à une *chaîne de caractères* ou vice-versa.

En Python, rien de plus simple de convertir un type de variable par un autre en utilisant ce que nous appelons en programmation une **fonction**. Une **fonction** (ou *function*) est une suite d'instruction que l'on peut appeler avec un nom (nous verrons cette notion en détails dans le prochain chapitre).

Prenons par exemple les trois fonctions de conversion en Python suivantes :

- **int()** : fonction qui renvoie une valeur *entière* (integer) à partir d'une variable de tous types ;
- **float()** : fonction qui envoie un nombre à virgule flottante (*floating point*) à partir d'une variable de tous types ;
- **str()** : fonction qui convertit la valeur spécifiée en une *chaîne de caractères* à partir d'une variable de tous types.

Pour vous en convaincre, appliquons ces fonctions dans les cas suivants :

<pre>1 i = 3 2 str(i)</pre> '3'	<pre>1 float(i)</pre> 456.0
<pre>1 i = '456' 2 int(i)</pre> 456	<pre>1 i = '3.1416' 2 float(i)</pre> 3.1416

On verra plus tard que ces **conversions** sont essentielles. En effet, lorsqu'on lit ou écrit des nombres, ils peuvent être considérés comme du **texte**, donc des chaînes de caractères. Toute conversion d'une variable d'un **type** en un autre est appelé *casting* en anglais, il se peut que vous croisie ce terme si vous consultez d'autres ressources.

2.7. Note sur la division de deux nombres entiers

Notez bien qu'en Python 3, la division de deux nombres *entiers* renvoie par défaut un *float* :

```
1 x = 3/4
2 x
```

0.75

```
1 type(x)
```

float

2.8. Note sur le vocabulaire et la syntaxe

- Nous avons vu dans ce chapitre la notion de **variable** qui est commune à tous les **langages de programmation**.
- Toutefois, Python est un langage dit « **orienté objet** », il se peut que dans la suite de votre apprentissage, vous employiez le mot **objet** pour désigner une **variable**. Par exemple, « une **variable** de type *entier* » sera pour nous équivalent à « un **objet** de type *entier* ».
- Par ailleurs, nous avons rencontré plusieurs fois des **fonctions** dans ce chapitre, notamment avec `int()`, `float()` et `str()`. On reconnaît qu'il s'agit d'une fonction car son nom est suivi de parenthèses (par exemple `int()`).

En Python, la syntaxe générale est *fonction()*. Ce qui se trouve entre les parenthèses d'une fonction est appelé **argument** et c'est ce que l'on « passe » à la **fonction**.

Pour l'instant, on retiendra qu'une **fonction** est une sorte de boîte à qui on passe un (ou plusieurs) **argument(s)**, qui effectue une action et qui peut renvoyer un résultat ou plus généralement un objet.

Si ces notions vous semblent obscures, ne vous inquiétez pas, au fur et à mesure que vous avancerez dans le cours, tout deviendra limpide.

3. Affichage

3.1. La fonction print()

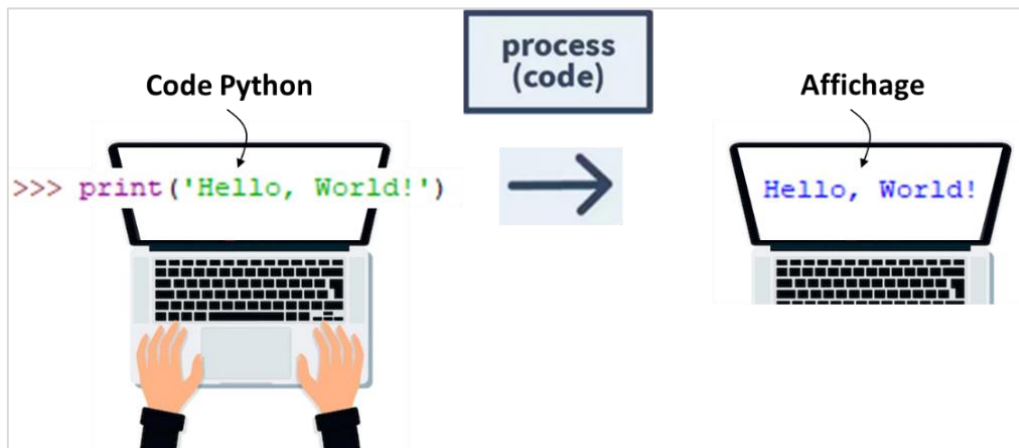
3.1.1. Application

Précédemment, nous avons rencontré la **fonction** `print()` qui affiche une *chaîne de caractères* (le fameux "Hello world!").

En fait, la fonction `print()` affiche l'**argument** qu'on lui passe entre **parenthèses** et un **retour à ligne**. Ce **retour à ligne** supplémentaire est ajouté par défaut :

```
1 print("Hello World!")  
Hello World!
```

Cette opération d'affichage à partir du codage de la fonction **fonction** `print()` est résumée graphique sur la figure ci-dessous..



Programmation et exécution de la fonction `print()`, suivi de l'affichage sur écran.

3.1.2. Paramétrage de la fonction par « mot-clé »

Si toutefois, on ne veut pas afficher ce **retour à la ligne**, on peut utiliser l'**argument** par « mot-clé » **end** :

```
1 print("Hello World!", end = "")  
Hello World!
```

Ligne 1 :

- On a utilisé l'instruction `print()` classiquement en passant la *chaîne de caractères* "Hello world!" en **argument**.
- On a ajouté un **second argument** `end=""`, en précisant le **mot-clé end**. Nous aborderons les **arguments** par **mot-clé** dans le Chapitre Fonctions. Pour l'instant, dites-vous que cela modifie le comportement par défaut des **fonctions**.

Affichage :

- L'effet de l'**argument** `end=""` est que l'on indique une fin après la *chaîne de caractères* "Hello world!", et non un **retour à la ligne** comme la fonction `print()` l'exécute par défaut

Une autre manière de s'en rendre compte est d'utiliser deux fonctions `print()` à la suite. Dans la portion de code suivante, le caractère « ; » sert à séparer plusieurs instructions Python sur une même ligne :

```
1 print("Hello") ; print("Joe")
Hello
Joe

1 print("Hello", end = "") ; print("Joe")
HelloJoe

1 print("Hello", end = " ") ; print("Joe")
Hello Joe
```

3.1.3. Autres paramétrages

La fonction `print()` peut également afficher le contenu d'une variable quel que soit son type. Par exemple, pour un entier :

```
1 var = 3
2 print(var)
3
```

Il est également possible d'afficher le contenu de plusieurs **variables** (quel que soit leur type) en les séparant par des **virgules** :

```
1 x = 32
2 nom = "John"
3 print(nom, "a", x, "ans")
John a 32 ans
```

Python a écrit une phrase complète en remplaçant les **variables** `x` et `nom` par leur contenu. Vous remarquerez que pour afficher plusieurs éléments de texte sur une seule ligne, nous avons utilisé le séparateur « , » entre les différents éléments.

Python a également ajouté un espace à chaque fois que l'on utilisait le séparateur « , ». On peut modifier ce comportement en passant à la fonction `print()` l'argument par mot-clé `sep` :

```
1 x = 32
2 nom = "John"
3 print(nom, "a", x, "ans", sep = "-")
John-a-32-ans
```

Pour afficher deux *chaînes de caractères* l'une à côté de l'autre, sans espace, on peut soit les concaténer, soit utiliser l'**argument** par mot-clé **sep** avec une *chaîne de caractères* vide :

```
1 ani1 = "chat"
2 ani2 = "souris"
3 print(ani1, ani2)

chat souris
```

```
1 print(ani1 + ani2)

chatsouris
```

```
1 print(ani1, ani2, sep = "")

chatsouris
```

3.2. Ecriture formatée

3.2.1. C'est quoi une « écriture formatée » ?

Que signifie « écriture formatée » ?

Définition

L'**écriture formatée** est un mécanisme permettant d'afficher des **variables** avec un certain **format**, par exemple justifiées à gauche ou à droite, ou encore avec un certain nombre de *décimales* pour les *floats*.

L'écriture formatée est incontournable lorsqu'on veut créer des fichiers organisés en « belles colonnes ».

3.2.2. Introduction aux f-strings

Depuis la version 3.6, Python a introduit les **f-strings** pour mettre en place l'écriture formatée que nous allons décrire en détail dans cette rubrique. Il existe d'autres manières pour formater des chaînes de caractères qui étaient utilisées avant la version 3.6, nous en avons mis un rappel bref dans la rubrique suivante. Toutefois, nous recommandons vivement l'utilisation des f-strings si vous débutez l'apprentissage de Python.

Que signifie « **f-strings** » ?

Définition

f-string est le diminutif de *formatted string literals*. Mais encore ? Précédemment, nous avons vu les *chaînes de caractères* ou encore *strings* qui étaient représentées par un **texte** entouré de **guillemets simples** « `{}` » ou **doublets** « `{}` ». Par exemple :

```
1 "Ceci est une chaîne de caractères"
```

L'équivalent en **f-string** est tout simplement la même chaîne précédée du caractère « **f** » sans espace entre les deux :


```
1 f"Ceci est une chaîne de caractères"
```

Ce caractère « f » avant les guillemets va indiquer à Python qu'il s'agit d'une **f-string** permettant de mettre en place le mécanisme de l'écriture formatée, contrairement à une *string* normale.

3.2.3. Prise en main des f-strings

Les **f-strings** permettent une meilleure organisation de l'**affichage des variables**. Reprenons l'exemple ci-dessus à propos de notre ami *John* :

```
1 x = 32
2 nom = "John"
3 print(f"{nom} a {x} ans")
```

John a 32 ans

Il suffit de passer un **nom de variable** au sein de chaque **couple d'accolades « {} »** et Python les remplace par leur contenu ! Première remarque, la syntaxe apparaît plus lisible que l'équivalent vu ci-avant `print(nom, "a", x, "ans")`.

Bien sûr, il ne faut pas omettre le caractère « f » avant le premier guillemet, sinon Python prendra cela pour une *chaîne de caractères* normale et ne mettra pas en place ce mécanisme de remplacement :

```
1 print("{nom} a {x} ans")
```

{nom} a {x} ans

Remarque

Une variable est utilisable plus d'une fois pour une f-string donnée :

```
1 print("{nom} a {x} ans")
```

{nom} a {x} ans

Enfin, il est possible de mettre entre les **accolades** des valeurs numériques ou des *chaînes de caractères* :

```
1 print(f"J'affiche l'entier {10} et le float {3.14}")
```

J'affiche l'entier 10 et le float 3.14

```
1 print(f"J'affiche la chaîne {'Python'}")
```

J'affiche la chaîne Python

Même si cela ne présente que peu d'intérêt pour l'instant, il s'agit d'une commande Python parfaitement valide. Nous verrons des exemples plus pertinents par la suite. Cela fonctionne avec n'importe quel type de variable (*entiers, chaînes de caractères, floats, etc.*). Attention toutefois pour

les *chaînes de caractères*, utilisez des guillemets simples au sein des **accolades** « `{}` » si vous définissez votre **f-string** avec des guillemets doubles.

3.2.4. Spécialisation de format

Les **f-strings** permettent de remplacer des variables au sein d'une *chaîne de caractères*. On peut également spécifier le format de leur affichage.

Prenons un exemple. Imaginez maintenant que vous vouliez calculer, puis afficher, la proportion de GC d'un génome. La proportion de GC s'obtient comme la somme des bases Guanine (G) et Cytosine (C) divisée par le nombre total de bases (A, T, C, G) du génome considéré. Si on a, par exemple, 4500 bases G et 2575 bases C, pour un total de 14800 bases, vous pourriez procéder comme suit (notez bien l'utilisation des parenthèses pour gérer les priorités des opérateurs) :

```
1 prop_GC = (4500 + 2575) / 14800
2 print("La proportion de GC est", prop_GC)

La proportion de GC est 0.4780405405405405
```

Le résultat obtenu présente trop de décimales (seize dans le cas présent). Pour écrire le résultat plus lisiblement, vous pouvez spécifier dans les accolades `{}` le format qui vous intéresse. Dans le cas présent, vous voulez formater un *float* pour l'afficher avec deux puis trois décimales :

```
1 print(f"La proportion de GC est {prop_GC:.2f}")

La proportion de GC est 0.48
```

```
1 print(f"La proportion de GC est {prop_GC:.3f}")

La proportion de GC est 0.478
```

Détaillons le contenu des accolades de la première ligne (`{prop_GC:.2f}`) :

- D'abord on a le nom de la variable à formater, `prop_GC`, c'est indispensable avec les f-strings.
- Ensuite on rencontre les **deux-points** « `:` », ceux-ci indiquent que ce qui suit va spécifier le format dans lequel on veut imprimer la variable `prop_GC`.
- À droite des deux-points on trouve « `.2f` » qui indique ce format : la lettre **f** indique qu'on souhaite afficher la variable sous forme d'un *float*, les caractères « `.2` » indiquent la précision voulue, soit ici deux chiffres après la virgule.

Notez enfin que le formatage avec « `.xf` » (`x` étant un *entier* positif) renvoie un résultat arrondi. Vous pouvez aussi formater des *entiers* avec la lettre « `d` » (ici « `d` » veut dire *decimal integer*) :

```
1 nb_G = 4500
2 print(f"Ce génome contient {nb_G:d} guanines")

Ce génome contient 4500 guanines
```

Enfin, il est possible de préciser sur combien de caractères vous voulez qu'un résultat soit écrit et comment se fait l'alignement (à gauche, à droite ou centré). Dans la portion de code suivante, le caractère `;` sert de séparateur entre les instructions sur une même ligne :

```

1 print(10) ; print(1000)

10
1000

1 print(f"{10:>6d}") ; print(f"{1000:>6d}")

      10
     1000

1 print(f"{10:^6d}") ; print(f"{1000:^6d}")

      10
     1000

1 print(f"{10:*^6d}") ; print(f"{1000:*^6d}")

**10**
*1000*

1 print(f"{10:0^6d}") ; print(f"{1000:0^6d}")

001000
010000

```

Notez que « > » spécifie un alignement à droite, « < » spécifie un alignement à gauche et « ^ » spécifie un alignement centré. Il est également possible d'indiquer le caractère qui servira de remplissage lors des alignements (l'espace est le caractère par défaut).

Remarque

Vous voyez tout de suite l'énorme avantage de l'écriture formatée. Elle vous permet d'écrire en colonnes parfaitement alignées.

3.2.5. Expressions dans les f-strings

Une fonctionnalité extrêmement puissante des **f-strings** est de supporter des expressions générales au sein des accolades. Ainsi, il est possible d'y mettre directement une opération ou encore un appel à une fonction :

```

1 print(f"Le résultat de 5 * 5 vaut {5 * 5}")

Le résultat de 5 * 5 vaut 25

1 print(f"Résultat d'une opération avec des floats : {(4.1 * 6.7)}")

Résultat d'une opération avec des floats : 27.47

```

```
1 entier = 2
2 print(f"Le type de {entier} est {type(entier)}")
```

Le type de 2 est <class 'int'>

3.3. Ecriture scientifique

Pour les nombres très grands ou très petits, l'**écriture formatée** permet d'afficher un nombre en notation scientifique (sous forme de puissance de 10) avec la lettre **e** :

```
1 print(f"{1_000_000_000:e}")
```

1.000000e+09

```
1 print(f"{0.000_000_001:e}")
```

1.000000e-09

Il est également possible de définir le nombre de chiffres après la virgule. Dans l'exemple ci-dessous, on affiche un nombre avec aucun, 3 et 6 chiffres après la virgule :

```
1 avogadro_number = 6.022_140_76e23
2 print(f"{avogadro_number:.0e}")
```

6e+23

```
1 print(f"{avogadro_number:.3e}")
```

6.022e+23

```
1 print(f"{avogadro_number:.6e}")
```

6.022141e+23

4. Exercices

4.1. Variables

Déclarer les trois variables suivantes :

- **nom** contenant ton prénom,
- **age** contenant ton âge,
- **ville** contenant la ville où tu habites.

Puis afficher la phrase :

```
Je m'appelle <nom>, j'ai <age> ans et j'habite à <ville>.
```

4.2. Opérations

Déclarer deux nombres :

a = 15 et **b = 4**

Calculer et afficher les résultats suivants :

1. La somme
2. La différence
3. Le produit
4. Le quotient
5. Le reste de la division entière
6. La puissance **a^b**

4.3. Conversion Celsius / Fahrenheit

Écrire un programme qui convertit une température donnée en degrés Celsius dans une variable **celsius** en Fahrenheit, dont la formule de conversion est :

$$F = \frac{9}{5} \times C + 32$$

Afficher la conversion sous la forme :

```
20°C = 68°F
```

4.4. Formatage de nombre

Soit le nombre π :

pi = 3.141592653589793

Afficher **pi** :

1. Avec 2 chiffres après la virgule
2. Avec 5 chiffres après la virgule
3. En notation scientifique

4.5. Mini-calculatrice

Ecrire un programme qui demande à l'utilisateur deux nombres entiers x et y . Pour cela, on utilisera la fonction `input()`.

Une fois les deux nombres entiers x et y saisis, afficher le résultat des 4 opérateurs de base (+, -, *, /) avec un formatage clair.

La fonction `input()` en Python est une fonction **intégrée** (builtin) qui permet à un programme de **lire une donnée saisie par l'utilisateur au clavier** pendant l'exécution.

Fonctionnement :

- Quand Python rencontre `input()`, il **suspend l'exécution** et attend que l'utilisateur tape quelque chose au clavier.
- Le texte saisi est **toujours renvoyé sous forme de chaîne de caractères** (`str`), même si l'utilisateur entre un nombre.

Syntaxe :

```
python

variable = input("Message à afficher : ")
```

Exemple simple :

```
python

nom = input("Quel est ton nom ? ")
print("Bonjour,", nom)
```

➡ Si l'utilisateur tape `Alice`, le programme affichera :

```
Bonjour, Alice
```

4.6. Conversions de types

Prédire le résultat de chacune des instructions suivantes, puis vérifier le résultat par programmation sur Jupyter Notebook.

```
— str(4) * int("3")
— int("3") + float("3.2")
— str(3) * float("3.2")
— str(3/4) * 2
```

Corrections

4.1. Variables

```
nom = "Ali"
age = 25
ville = "Paris"

print(f"Je m'appelle {nom}, j'ai {age} ans et j'habite à {ville}.")
```

4.2. Opérations

```
a = 15
b = 4

print(f"Somme : {a + b}")
print(f"Différence : {a - b}")
print(f"Produit : {a * b}")
print(f"Quotient : {a / b}")
print(f"Division entière : {a // b}")
print(f"Reste : {a % b}")
print(f"Puissance : {a ** b}")
```

4.3. Conversion Celcius /Fahrenheit

```
celsius = 20
fahrenheit = (9/5) * celsius + 32

print(f"{celsius}°C = {fahrenheit}°F")
```

4.4. Formatage de nombre

```
pi = 3.141592653589793

print(f"Pi avec 2 décimales : {pi:.2f}")
print(f"Pi avec 5 décimales : {pi:.5f}")
print(f"Pi en notation scientifique : {pi:.2e}")
```


4.5. Mini-calculatrice

```
x = int(input("Entrez le premier nombre : "))
y = int(input("Entrez le deuxième nombre : "))

print(f"{x} + {y} = {x + y}")
print(f"{x} - {y} = {x - y}")
print(f"{x} * {y} = {x * y}")
print(f"{x} / {y} = {x / y:.2f}")
```

4.6. Conversions de types

A vérifier par soi-même !